DA3408 A4: Application Monitoring — Observability with Prometheus & Grafana

Objective: Instrument a CPU-bound AI application, configure an alerting pipeline, and design a production-grade monitoring dashboard.

1. The Application: CPU-Only Image Captioning

Build a **Streamlit** application that utilizes a lightweight, CPU-friendly model (e.g., `blip-image-captioning-base` or `vit-gpt2-image-captioning`).

Core Features:

- **Single Mode:** Upload an image and generate a caption.
- **Bulk Mode:** Upload a ZIP file of images for batch processing.
- **Instrumentation:** Use the `prometheus_client` library to expose a `/metrics` endpoint on a separate port.

2. Monitoring Stack

You are required to set up the following infrastructure:

A. Instrumentation (Python)

Define and track the metric types listed below with examples. Create as many metrics as appropriate. **The more the better.** Additionally, add custom labels (a.k.a attribute-value pairs) to enrich the metrics.

1. **Counters:** Ex. Total images processed (categorized by Single vs. Bulk).
2. **Gauges:** Ex. Current number of active requests or memory usage of the model.
3. **Histograms:** Ex. Latency/Inference time per image.
4. **Summary**: Generate metrics with _sum and _total suffixes for suitable measures.
5. **Custom Labels:** Track client metadata (e.g., source IP or session ID).

B. Infrastructure Metrics

Deploy and configure **node_exporter** on your machine. Ensure Prometheus scrapes both your AI application metrics and the system-level node metrics (CPU/RAM/Disk) to correlate application spikes with hardware usage. **The corroboration between system and application metrics is crucial.**

D. Grafana Dashboard

Create a real-time dashboard that adheres to the **7 Commandments of Plotting**. It must visualize all relevant metrics, either singularly or in multiple views. A few of the mandatory items are listed below, **but not limited to.**

- Application health (Up/Down status).
- Throughput and Latency.

- System resource utilization (correlated with bulk uploads).

---

3. Reading Material & Resources

- **Streamlit:** Official Documentation - Main Concepts
- **Prometheus:** Python Client Library Guide
- **Grafana:** Best Practices for Creating Dashboards

---

4. Code of Conduct: Fair Use of AI

- **The "Black Box" Metric Rule:** You may use AI to generate the PromQL queries for your dashboard. However, you must manually calculate the metric's logic during evaluation.
- **Attribution:** Any complex Regex or PromQL generated by AI must be commented in your configuration files with the prompt used.

---

5. Grading Rubric (60 Points)

Category	Weight	Criteria
Python Instrumentation	25	Correct use of Counters, Gauges, Summary, and Histograms; effective use of labels for client tracking.
Node Exporting	5	Successful integration of `node_exporter` metrics into the Prometheus scrape config.
Grafana Intuitiveness	20	Adherence to plotting commandments; logical flow from high-level health to low-level resource metrics.
Application Logic	10	Streamlit app handles Single and Bulk (ZIP) uploads without crashing on the CPU.

Final Deliverable

- **The Code:** GitHub repo with Streamlit app, `prometheus.yml`.
- **The Dashboard:** A JSON export of the Grafana Dashboard + video recording of your dashboard for about a minute with 5 seconds refresh rate.

6. Seven Commandments

- The color palette should be inclusive of the color-challenged audience.
- Markers should remain relevant across media (print, browser, PDF, etc.).
- Axes should be named appropriately.
- Scale (lin/log) and units of the axes should be explicitly specified.
- A legend should be present when more than one variable is visualized.
- A brief title (and sub-title) for the plot should be provided.
- A short description of the plot's story to make it self-explanatory.

Remember, your plots should speak for themselves

even in your absence!